Pasqal's challenge

Analog quantum devices for optimization problems

At a glance

Problem	Capacitated Vehicle Routing (CVRP), solved by the cluster-first / route-second decomposition.
Track A (intermediate)	Capacity-feasible clustering of customers onto vehicles.
Track B (advanced)	Clustering and intra-cluster routing (TSP), end-to-end.
The challenge	Every extra binary variable costs you an atom. The goal is to formulate the problem with the fewest variables, to get the best solutions out of the hardware and to use the quantum computation as the core of the optimization, so that the classical parts stay computationally light.

Note about the document

This document describes the problem, the machine, and what you must deliver, in words. Writing down the cost function, choosing the decision variables, encoding the constraints, and deciding what to do about the ones that do not fit is part of the challenge.

There is more than one defensible model. We are interested in the one you chose, why you chose it, and what it cost you.

On the use of AI

Use it. Coding assistants and language models are part of how this work gets done now. Use them for code, for debugging, for plots, for finding your way into an unfamiliar literature, for turning your derivation into LaTeX.

What we will not accept is unreviewed output. You own every line you submit and every claim you make about it. At judging, we may ask any member of your team to explain any part of your submission, re-derive a penalty term, justify a weight, say what a function does. "The model wrote it" is not an answer, and a team that cannot explain its own model will score as though it did not have one.

This field is a particularly hard one to trust a model on. LLMs are fluent and confident about neutral-atom hardware and frequently wrong: they will hand you a cost function with negative couplings that cannot be embedded, a layout that violates the minimum atom spacing, a pulse longer than the device allows, or a citation to a paper that does not exist. Check everything against the device simulation itself, not against whether it sounds right.

And avoid slop. A report padded with fluent, generic paragraphs about the promise of quantum optimization is worth less to us than three honest sentences about what broke. Five encodings nobody on the team understands are worth less than one you can defend. Say in your report where you used AI and what for. There is no penalty for answering "everywhere", and there is one for pretending otherwise.

Contents

1 Why this problem **2**

2 The problem, in words **2**

2.1 Instance tiers . . . 3

2.2 The decomposition we will use . . . 3

3 Neutral atoms introduction **4**

3.1 Hardware envelope . . . 4

4 The core challenge: spend fewer variables **5**
4.1 Finding a way out . . . 5

5 Track A (intermediate): capacity-feasible clustering **6**
5.1 What you must build . . . 6

6 Track B (advanced): clustering and routing **6**
6.1 Encoding the Traveling Salesman Problem (TSP) . . . 6

7 Toolchain **7**

8 Embedding and pulse shaping **7**
8.1 Embedding . . . 8
8.2 Pulse shaping . . . 8

9 Evaluation **8**
9.1 Gap to optimality . . . 8
9.2 Which gaps to report . . . 8
9.3 Reference values . . . 9
9.4 Metrics you must report . . . 10

10 Deliverables **10**

1 Why this problem

The Capacitated Vehicle Routing Problem (CVRP) is one of the most important problems of logistics. A fleet of vehicles, each with a fixed capacity, leaves a central depot, serves a set of customers with known demands, and returns. The objective is to serve every customer exactly once, never exceed a vehicle's capacity, and minimize total travel distance.

Parcel carriers re-solve a CVRP every night to decide which packages go on which van and in what order. Grocery distributors face the same problem with weight and volume limits. Waste collection services face it with truck capacity; hospital supply chains face it with cold-chain constraints. The economic stakes are large: better routes mean less fuel, fewer driver-hours, less vehicle wear, and lower emissions.

The computational stakes are also large. CVRP is NP-hard, and exact solvers become impractical well before the instance sizes that appear in daily operations. This is precisely the regime where heuristics, and, potentially, quantum optimization, can help.

> **Quantum at the moment**
>
> No quantum device today beats a well-tuned classical CVRP solver. This is not what is being evaluated. You are being evaluated on how efficiently you can *encode* a constrained optimization problem into the native physics of a neutral-atom register, and how well you can extract feasible solutions from it.

2 The problem, in words

You are given a depot, a set of customers, and a fleet of vehicles.

Each customer has a known location and demand, the number of units they need delivered. Every vehicle is identical, starts at the depot, ends at the depot, and can carry at most a fixed number of units in total. Moving between any two locations has a known cost, which you may take to be the Euclidean distance between them.

A valid plan assigns every customer to exactly one vehicle, never loads a vehicle beyond its capacity, and specifies the order in which each vehicle visits its customers. The cost of a plan is the total distance driven by the whole fleet. You want the cheapest valid plan.

Two things about the problem are worth noticing for the challenge. First, "exactly one vehicle" is an *equality*, every customer served, none served twice. Second, "at most capacity" is an *inequality*, and a vehicle that runs half empty is perfectly valid. That asymmetry between the two kinds of constraint is where most of the difficulty in this challenge lives.

Assumptions. Throughout, we assume the following.

1. The depot location is specified before clustering and remains fixed throughout the optimization.
2. Travel costs are symmetric and satisfy the triangle inequality: going directly is never more expensive than going via a third location.
3. Each customer is served by a single vehicle in a single visit. Deliveries are not split.
4. Each vehicle makes exactly one trip: it leaves the depot once and returns once, and may not return to the depot mid-route to reload.
5. No customer's demand exceeds the vehicle capacity.
6. The fleet size K is fixed and part of the instance. A vehicle may be left unused. An unused vehicle travels no distance and costs nothing.

These assumptions do not guarantee that a valid plan exists. Total demand not exceeding total capacity is necessary but not sufficient: two vehicles of capacity 3 cannot serve three customers of demand 2. Deciding whether customers can be assigned to vehicles within capacity is itself a bin-packing problem. Since you generate your own instances (Task A2), it is your responsibility to ensure that every instance you report on has at least one valid plan, and to prove it.

Derive 2.1 — The problem

Before anything else, write the CVRP down formally: name your sets, your parameters, your decision variables, your objective, and your constraints. Be explicit about which constraints are equalities and which are inequalities. You will refer back to this all weekend, and every later modeling decision is a decision about how to distort it.

2.1 Instance tiers

Your instances must come from the three tiers below. Within a tier, you choose the seeds, customer locations, and individual demands, subject to Section 2's assumptions and Task A2's certification requirement.

Tier	Customers N	Vehicles K	Capacity Q	Demand range	Intended for
S	4	2	7	$\{1, \ldots, 4\}$	`pulser, emu-sv`
M	10	3	20	$\{1, \ldots, 8\}$	`emu-mps`
L	15	4	30	$\{1, \ldots, 10\}$	`emu-mps` or QPU

Table 1: Instance tiers. N, K, Q and the demand range are fixed; seeds, locations and individual demands are your choice.

Customer and depot locations lie in a square of side 100 distance units. The depot position is your choice but must be stated

2.2 The decomposition we will use

Solving the whole problem at once means deciding, for every customer, both which vehicle serves it and where in that vehicle's route it comes. The direct encoding uses one binary variable per (customer, vehicle, position) triple, so the

count grows on the order of N^2K before a single slack bit is added: 32 on tier S, 300 on tier M and 900 on tier L, against a budget of 80 atoms. Tier S fits within the 80-atom budget on paper, but at 32 variables it is already beyond exact emulation; tiers M and L are far beyond anything current analog devices can run.

So we split the problem in two. First, decide which customers each vehicle serves, ignoring order; this is a capacity-constrained clustering problem. Then, for each vehicle separately, decide the *order* in which it visits the customers it was given, a Traveling Salesman Problem on a handful of points. This is the "cluster-first, route-second" heuristic.

Pitfall 2.1 — The decomposition is not exact

Cluster-first / route-second is a heuristic. The optimal CVRP solution is not, in general, the best routing of the best clustering, because a clustering objective that ignores tour length can separate customers that a good route would visit consecutively.

3 Neutral atoms introduction

A Pasqal neutral atom device is an array of individual atoms held in optical tweezers. You choose where the atoms sit in a 2D plane. A global laser then drives every atom between its ground state $|g\rangle$ and a highly excited Rydberg state $|r\rangle$. In the analog (Ising) mode, the register evolves under

$$\frac{H(t)}{\hbar} = \sum_i \frac{\Omega(t)}{2}\,\hat{\sigma}_i^x - \delta(t)\sum_i \hat{n}_i + \sum_{i<j} \frac{C_6}{\|\mathbf{r}_i - \mathbf{r}_j\|^6}\,\hat{n}_i\hat{n}_j, \tag{1}$$

where t is the time dependency that sweeps between an initial and a final time for a specific duration, $\hat{n}_i = |r\rangle\langle r|_i$ counts whether atom i is excited, $\Omega(t)$ is the Rabi frequency set by the laser amplitude, $\delta(t)$ is the detuning, and $\mathbf{r}_i$ is the position you chose for atom i.

Each atom is either excited or not, so a measurement of the register returns a string of zeros and ones, i.e., an assignment of values to binary decision variables. Two of the three terms in Equation (1), $\delta(t)$ and $\frac{C_6}{\|\mathbf{r}_i-\mathbf{r}_j\|^6}$, do not mix those strings with each other: they simply assign an energy to each one. The third term, the one carrying Ω, is what lets the system explore.

If you prepare the system in its ground state and change Ω and δ slowly enough, the register stays in the ground state throughout and ends up in the lowest-energy bitstring of the final Hamiltonian. Measure it, and you have sampled a low-cost solution of whatever optimization problem those two diagonal terms happen to encode.

Derive 3.1 — What does the register actually minimize?

Write down, explicitly, the classical cost function that Equation (1) assigns to a bitstring once Ω has been ramped back to zero. You should recognize its form. Identify precisely which parts of it you control through the laser, which parts you control through the positions of the atoms, and which parts you do not control at all. Everything you do for the rest of the weekend is an attempt to make that cost function look like the CVRP.

3.1 Hardware envelope

Design against the `AnalogDevice` specification in Pulser; it is the model closest to the QPU you will submit to.

Parameter	Value	What it costs you
Max. atoms	80	Hard cap on total binary variables.
Min. atom separation	$5\,\mu$m	Upper bound on the largest realizable coupling.
Max. register radius	$38\,\mu$m	Bounds the register; unwanted couplings can be made small.
Max. Rabi frequency $\Omega_{\max}$	$12.57\,\text{rad}/\mu\text{s}$ ($2\pi \times 2\,\text{MHz}$)	Sets R_b at full power.
Detuning range	$\pm 125.66\,\text{rad}/\mu\text{s}$ ($2\pi \times 20\,\text{MHz}$)	Sets the range of the shared linear coefficient
Max. sequence duration	6000 ns	Bounds how adiabatic you can afford to be.
$C_6/\hbar$	$8.657 \times 10^5\,\text{rad}/\mu\text{s}\cdot\mu\text{m}^6$	Fixed. Sets the coupling scale.

The 80-atom cap makes *slack variables* costly, as discussed in the next section 4: *Each slack bit uses an atom that could be allocated elsewhere.* The detuning of `AnalogDevice` is global: every atom sees the same $\delta(t)$. In emulation

only, you may add a detuning map modulator (DMM), which applies an extra detuning $\varepsilon_i\,\delta_{\mathrm{DMM}}(t)$ to atom i, with fixed weights $\varepsilon_i \in [0,1]$ and $\delta_{\mathrm{DMM}}(t) \leq 0$. Results that use the DMM must be labeled as such and are compared separately from global-detuning results. QPU runs use global detuning only.

Note: During the hackathon, no quantum hardware will be used. You will solve the challenge using only the **emulators (EMUs)** due to time constraints and to ensure fair competition. Afterward, some teams may have access to the QPU as a prize.

4 The core challenge: spend fewer variables

Analog neutral-atom devices minimize an unconstrained cost function. The CVRP is a constrained problem. Bridging that gap is the core challenge, and the standard bridge is expensive.

The textbook approach to a constraint is to fold it into the objective as a penalty: add a term that is zero when the constraint holds and large and positive when it does not, weighted heavily enough that violating it is never worth the saving elsewhere. For *equality* constraints, this is straightforward. For *inequality* constraints it is not, because there is no single target value to penalize deviation from. A vehicle may be loaded anywhere between empty and full.

The conventional solution is to introduce a **slack variable**: a non-negative auxiliary quantity that absorbs the difference between the actual load and capacity, turning the inequality into an equality that can be penalized as usual. Because the device supports only binary variables, the slack quantity must also be encoded in bits, requiring several bits per vehicle. The squared penalty couples each slack bit to every assignment variable, creating exactly the dense, long-range interaction pattern that a flat atomic register handles worst.

So slack variables hurt you twice: they consume atoms from a budget of 80, and they degrade the structure of the problem you then have to embed.

Derive 4.1 — The cost of the obvious approach

Work out the textbook formulation: penalty terms for both the assignment constraint and the capacity constraint, with binary-encoded slack. Then count. Fill in this table for each tier as an example.

Encoding	Variable count	S	M	L	Fits in 80 atoms?
Assignment variables only, no constraints	...	...	...	...	...
+ binary-encoded slack	...	...	...	...	...
Your formulation	...	...	...	...	...

4.1 Finding a way out

Slack is the textbook answer, not the only one. Inequality constraints are a recurring nuisance in quantum optimization; they show up in knapsack problems, in scheduling, in portfolio selection, in every routing problem anyone has tried to put on a quantum device. And people have been publishing ways around them for years. Some avoid auxiliary variables by changing the shape of the penalty. Some avoid the inequality altogether by changing what the model asks for. Some change how a choice between several options is represented, which turns out to matter as much for the coupling structure as for the variable count.

We are not going to tell you which of these to use, or name them. Finding the relevant literature, judging which approach fits the model you derived, deriving the formulation, and measuring what it costs you *is* the challenge.

A warning about what you will find. Most of this literature is written for quantum annealers or for gate-based QAOA, where the coupling graph is either fixed hardware or freely programmable. Neither is true here: your couplings come from distances in a plane, they are all the same sign, and there are only so many atoms you can put near one another. An approach that halves the variable count but produces a coupling pattern you cannot lay out is not a saving.

5 Track A (intermediate): capacity-feasible clustering

5.1 What you must build

Assign every customer to exactly one vehicle, such that no vehicle's total demand exceeds its capacity and the customers sharing a vehicle are close to one another. "Close" can be measured in more than one way. Distance to a cluster center, total pairwise distance within a cluster, or something of your own. And the choice matters, both for solution quality and for how hard the result is to embed.

Task 5.1 — A1 The clustering model

Choose your decision variables and write the clustering objective and both constraints. Then convert to the unconstrained form the device needs, and state clearly which penalty weights you introduced and how you will set them.

Task 5.2 — A2 Generate instances

Build an instance generator. Every instance must:

- be reproducible from a seed, with the generator code and seeds included in your submission
- satisfy all assumptions of Section 2
- be certified feasible: ship each instance with one valid customer-to-vehicle assignment as a certificate, which the validator checks;
- report its load ratio $\rho = \sum_i d_i/(K \times Q)$ and its largest-demand ratio d_{max}/Q. Here, d_i denotes the demand of customer i, and $d_{\max}$ is the largest customer demand in the instance.

How you guarantee feasibility is your choice. You may construct a feasible assignment first, solve the assignment problem exactly and reject infeasible instances, or enforce a sufficient condition such as $\sum_i d_i \leq K(Q - d_{max} + 1)$, under which any greedy assignment succeeds. State which you used.

Task 5.3 — A3 — Embed

Map your formulation onto atom positions respecting the minimum separation and maximum register radius. Report how far the interactions you actually realized are from the ones you wanted.

Task 5.4 — A4 Shape the pulse

Design the amplitude and detuning pulses. Run them on multiple instances. Report the results.

6 Track B (advanced): clustering and routing

Track B is Track A plus the second half of the pipeline: given the customers assigned to one vehicle, find the cheapest order in which to visit them, starting and ending at the depot.

6.1 Encoding the Traveling Salesman Problem (TSP)

Ordering is harder to encode than grouping. Grouping asks which set each customer belongs to. Ordering asks which *position in a sequence* each customer occupies, and the cost of a tour depends on which customers are *adjacent* in that sequence rather than on which ones share a label.

Pitfall 6.1 — Cluster sizes are not bounded

The tiers bound the number of customers, not the size of a cluster. A single vehicle can carry up to $\min(N, \lfloor Q/d_{\min} \rfloor)$ customers, where $d_{\min}$ is the smallest customer demand in the instance: as many as 10 on tier M and 15 on tier L when some demands are small. An encoding that assigns each customer to a position in the tour needs on the order of n^2 variables for a cluster of n customers, so 25 for $n = 5$ and 100 for $n = 10$. Your clustering will usually produce smaller clusters than that, but nothing guarantees it, and the optimum of your clustering model may not either. Decide early what you will do with a cluster your routing encoding cannot run, rather than discovering it on your largest instance. Section 9.2 says how such clusters are reported.

Task 6.1 — B1 The routing model

Write down the TSP for one cluster: decision variables, objective, and the constraints that make a valid tour.

Task 6.2 — B2 Encode the routing problem with fewer variables

Implement at least two encodings of the intra-cluster routing problem and compare them. Compare on variable count and embeddability.

Task 6.3 — B3 Close the loop

Run the full pipeline on multiple instances: quantum clustering, then quantum routing per cluster, then merged solution and total cost. Report the routing, end-to-end, and decomposition gaps of Section 9.2, and the split into decomposition, clustering, and routing loss. For each instance, also report how many clusters were trivial and how many were routed classically (Section 9.2).

7 Toolchain

Layer	Library	Use
Pulse-level	`pulser`	Registers, channels, waveforms, sequences. The ground truth for what is physically legal.
Abstract	`qoolqit`	Dimensionless program design plus register and embedding utilities, compiled down to a device afterward. Recommended if you want to think about the algorithm before the hardware.
Solver	`qubo-solver`	High-level pipeline, with classical solvers for comparison.
Emulation	`pulser-simulation`	Exact QuTiP state-vector evolution. Comfortable to about 12 atoms.
	`emu-sv`	State-vector emulator. Faster, up to about 20 atoms.
	`emu-mps`	Tensor-network (MPS) emulator. Reaches about 30-60 atoms when entanglement stays moderate (bond dimension). This is what you will need for bigger instances.
Hardware	`pasqal-cloud`	QPU and cloud-emulator submission.

Pitfall 7.1 — Emulators

`emu-mps` is approximate: it truncates entanglement at a user-selected bond dimension. For results on large instances, report the emulation parameters, including the bond dimension.

8 Embedding and pulse shaping

Two steps sit between the model you derived and a measured bitstring. *Embedding* decides where the atoms go. *Pulse shaping* decides what the laser does. Both will quietly ruin your results if you get them wrong, so it is worth understanding what they are before you hand them to a library.

8.1 Embedding

Your model is a cost function over binary variables: a linear coefficient for each variable, and a coupling strength for each pair. The register can reproduce only part of that. Couplings come from the distances between atoms, so they are all positive, they fall off as the sixth power of the distance, and no pair can be made exactly uncoupled. Linear terms come from the detuning, and with a global detuning every atom sees the same one: the register can only realize a cost function in which every variable has the same linear coefficient. In emulation, the DMM relaxes this in one direction only: since $\varepsilon_i \in [0, 1]$ and $\delta_{\mathrm{DMM}} \leq 0$, it can make exciting selected atoms less favorable than the rest, never more. Embedding is therefore the problem of finding atom positions whose interactions match the couplings your model asked for, and a final detuning that matches its linear terms, while respecting the device's minimum separation and maximum radius.

You will not match them exactly. Embedding is an approximation; it has an error, and that error is part of your result rather than an implementation detail.

8.2 Pulse shaping

The standard recipe is adiabatic. Begin at a large negative detuning, where the lowest-energy state is the one with no atom excited. Ramp the laser amplitude up, sweep the detuning to the final value your embedding requires (Section 8.1), then ramp the amplitude back down. If the sweep is slow compared with the energy gap between the ground state and the first excited state, the register stays in its ground state throughout and finishes in the bitstring that minimizes your cost function. Measure it, and you have a solution.

The baseline is a smooth three-point ramp for the amplitude and a linear sweep for the detuning, over a few microseconds. Everything you can do to improve on it comes down to two questions: what the amplitude and detuning scales should be, and where in the sweep you should slow down.

One thing to keep in mind while optimizing: you do not need the device to produce good answers on average, only to produce one good answer. Scoring a candidate pulse by the mean measured cost therefore throws away most of what the shots told you. Instead, score it by the metrics in Section 9.4 that reward good shots: the best gap, the probability of reaching the reference, and the feasibility rate.

> **Where to get the details**
>
> Both steps have library support
>
> - `qubo-solver`: a reference end-to-end pipeline that takes a cost function and returns solutions, embedding and pulse included. The fastest way to get a first result, and a sensible baseline to measure your own work against.
> - `qoolqit`: registers, embedders, drives and waveforms in dimensionless form, compiled down to a real device afterward. Use it if you want to reason about the algorithm before committing to hardware numbers.

9 Evaluation

9.1 Gap to optimality

For a feasible solution of cost C on an instance with reference cost C^*, the gap to optimality is

$$\mathrm{gap} = \frac{C - C^*}{C^*} \times 100\,\%. \tag{2}$$

A gap of $0\,\%$ means the solution matches the reference. Gaps are computed on feasible solutions only.

When C^* is a best known solution rather than a proven optimum, a gap may be negative: you found a better solution than the reference. Report it as such; do not clip it to zero.

9.2 Which gaps to report

The gap can be measured against different problems. Report each of the following, labeled clearly. "Routed exactly" means each cluster is routed by an exact TSP solver. The CVRP reference value is obtained with the hierarchy of Section 9.3, applied to the full CVRP.

Gap	Cost C	Reference C^*	Track
Model gap	Your clustering objective on the best feasible quantum shot	Optimum of your constrained clustering model	A, B
CVRP gap	Quantum clustering, routed exactly	CVRP reference value	A, B
Routing gap	Total length of your quantum tours, non-trivial quantum-routed clusters only	Exact TSP on the same clusters	B
End-to-end gap	Quantum clustering and quantum routing	CVRP reference value	B
Decomposition gap	Optimum of your clustering model, routed exactly	CVRP reference value	B

Table 2: Gap levels.

In absolute cost, the excess of your end-to-end solution over the CVRP reference is the sum of three parts: the decomposition loss (your model's optimal clustering routed exactly, minus the reference), the clustering loss (your best shot's clusters routed exactly, minus your model's optimal clustering routed exactly) and the routing loss (your end-to-end cost, minus your best shot's clusters routed exactly). Track B teams report all three. The clustering loss can be negative: the optimum of your clustering model need not give the shortest routes.

Routing results are only comparable between clusters of the same size, so report them per cluster: its number of customers n, the routing method, the atom count, and its tour length against the exact TSP tour on the same cluster.

Providing insights on how much useful work the quantum part does is much appreciated, especially if classical pre- or post-processing is involved (e.g. by computing the gap of the pipeline against the same pipeline but replacing the quantum part with a random sampler).

Trivial clusters. With the depot fixed, a cluster of $n \geq 2$ customers has $n!/2$ distinct tours: one for $n = 2$, three for $n = 3$, twelve for $n = 4$. A cluster of $n = 1$ has exactly one tour. We call a cluster trivial if $n \leq 3$. Such clusters have so few tours that a random valid tour is optimal with probability at least $1/3$, so they say little about a routing encoding. Route trivial clusters however you like; they are excluded from the routing gap and from the probability of reaching the reference, but their tours still count towards the end-to-end cost. On tier S, nearly every cluster is trivial, so Track B results on tier S validate your pipeline but say nothing about your routing.

Clusters you cannot run. If a cluster is too large for your routing encoding on the device or emulator you used, you may route it classically. State the size threshold you applied and the classical method, label the cluster as classically routed in results.csv, and exclude it from the routing gap. Its tour still counts towards the end-to-end cost, but report alongside the end-to-end gap the fraction of non-trivial clusters that were routed quantumly. An end-to-end gap in which every hard cluster was routed classically is not a quantum routing result.

9.3 Reference values

Since you generate your own instances, you also compute their reference values. For each instance, C^* is obtained by the first applicable method:

1. **Brute force.** If the solution space is small enough, enumerate every feasible solution. C^* is the *proven optimum*. Also record the number of distinct optimal solutions.
2. **CPLEX.** Otherwise, solve with CPLEX under a stated time limit. C^* is the *best known solution*.
3. **Simulated annealing.** You can also run simulated annealing with stated parameters and multiple independent restarts. C^* is the best value found by either method and is recorded as the *best known solution*.

On tier S, run both brute force and CPLEX and check that they agree. A mismatch indicates a modeling or formulation error.

Using `qubo-solver` for CPLEX and simulated annealing. Pasqal's `qubo-solver` package provides both classical solvers behind the same interface as its quantum pipeline: set `use_quantum=False` in `SolverConfig` and choose `classical_solver_type="cplex"` or `"simulated_annealing"` in `ClassicalConfig`. It solves the penalized QUBO. Its optimum is a valid reference for your constrained model only if it decodes to a feasible solution. For the CVRP reference, solve the constrained problem directly, for example with CPLEX through its own Python API

You are free to use any other implementation instead, provided you report its settings.

Report the solver settings used: for CPLEX, the time limit (`cplex_maxtime`); for simulated annealing, the initial and final temperatures (`sa_initial_temp`, `sa_final_temp`), the cooling rate (`sa_alpha`), the iteration limit (`max_iter`), the time limit (`sa_time_limit`), the number of restarts and the seeds.

9.4 Metrics you must report

Metric	Definition	What it tells us
Best gap	Gap of the best feasible shot, at each level of Section 9.2	Solution quality.
Feasibility rate	Fraction of shots satisfying every constraint.	How well your encoding enforces the constraints.
Probability of reaching reference	Fraction of shots with gap $\leq \varepsilon$. Choose ε and report it.	What a longer or better sweep should improve.
Atom efficiency	Total atoms in the register.	Your variable-reduction score in one number.
Embedding fidelity	Relative error between realized and target couplings; state your formula.	How faithful the register is to your cost function.
Reference status	Proven optimum (brute force) or best known solution (CPLEX or SA).	How to read every gap reported.

Table 3: Metrics to report for every (instance, method, seed).

Pitfall 9.1 — Feasibility is not optional

Once you drop slack variables, a large fraction of your measured bitstrings will violate constraints. Decide up front and state clearly: do you report the best *feasible* bitstring, or the best bitstring after a classical post-processing step? Both are defensible. Reporting an infeasible bitstring's cost is not. Always report the feasibility rate alongside the gap.

Pitfall 9.2 — A gap is only as good as its reference

A gap of 0 % against a weak simulated-annealing run says little. If no shot is feasible, the gap is undefined: report it as such.

Pitfall 9.3 — Identical vehicles

Vehicles are identical, so a clustering with k non-empty clusters appears $K!/(K-k)!$ times under different vehicle labels: once for each way of assigning its k groups to distinct vehicles. When counting distinct optimal solutions or shots that reach the reference, treat solutions that differ only by a relabeling of vehicles as the same solution: compare clusterings as sets of customer groups

10 Deliverables

Deliverable 10.1 — Due at the submission deadline

1. **Repository** with a `README.md` giving a one-command reproduction of your headline result, and a pinned environment (`requirements.txt` or `environment.yml`).
2. **Model write-up**: your formulations, derived from scratch, with the variable-count table of Section 4 completed for your instances.
3. **Instances**: generator, seeds, feasibility certificates, validator, reference values.
4. **Results**: a `results.csv` with one row per (instance, method, seed) and the metrics of Section 9.
5. **Report** (max 4 pages): model, variable accounting, embedding, pulse design, results, and an explicit "what

did not work" section.

Hint 10.1 — Advice from previous hackathons

Teams fail in two opposite ways, and you should know which one you are drifting towards.

The first is starting to code immediately, on the first formulation anyone suggests, and discovering at hour thirty that it never fitted on the device. The first four hours of this challenge belong on a whiteboard.

The second is the opposite: spending twenty hours perfecting an elegant model and never getting a single bitstring out of an emulator. Once you have a model you believe in, build the ugliest possible end-to-end pipeline, smallest instance, crudest encoding, default drive, and only then improve it. A working baseline is the only way to know whether your later improvements help.